

4. OOP avanzado

Last edited time	October 14, 2026 8:55 PM
Practica	☐
Teoria	☑

Herencia

Polimorfismo

- Overloading de operadores en Python
- `__repr__` vs `__str__`
- *Duck typing*

Multiherencia

- Problema del diamante
- Método `__mro__`
- Cómo implementar correctamente multiherencia

Clases abstractas

Decoradores

- Objetos de primera clase
- Sintaxis de los decoradores
- Función `property` y decorador `@property`
- Métodos estáticos

Diagrama de clases

- Elementos de un diagrama de clases
  - Clases
  - Relaciones

Metaclases

- Creación de metaclases
  - Método `__new__`
  - Método `__init__`
- Creación de instancias



- `isinstance(objeto, clase)` retorna True si `objeto` es una instancia de `clase`
- `hasattr(objeto, "nombre_atributo")` retorna True si `objeto` tiene un atributo llamado `nombre_atributo`

Herencia

- La herencia es una relación de **especialización** y **generalización** entre clases. De tal forma, que una clase (**subclase**) hereda atributos y métodos de otra (**superclase**).
  - La subclase posee todos los métodos de la superclase, pero además tiene métodos/atributos propios, que se consideran una especialización.

▼ Terminología

- Si `FurgonEscolar` **hereda** de `Auto` , también se dice que:
  - `FurgonEscolar` es una **especialización** de la clase `Auto`
  - `FurgonEscolar` es una **subclase** (o clase hija) de `Auto`

- `FurgonEscolar` **extiende** a la clase `Auto`
- `Auto` es **superclase** (o clase madre) de `FurgonEscolar`

▼ Para **crear una subclase** hay que especificar la superclase en la declaración de la clase

```
class SubClase(SuperClase):

# Esto crea una SubClase con todos los atributos y métodos
# que tiene la SuperClase.
```

▼ Si queremos añadir atributos propios o queremos modificar cómo se inicializa la clase, debemos modificar el método `__init__`

```
class SubClase(SuperClase):
    def __init__(self, atrSpC1, atrSpC2, atrSbC):
            # La clase almacena los atributos de la SuperClase:
        SuperClase.__init__(self, atrSpC1, atrSpC2)
        # Agrega un atributo propio:
        self.atributo_propio = atrSbC

    # Agrega un método propio:
    def metodo_propio(self,n):
        return n**2
```

`SubClase` va a tener todos los atributos y métodos de `SuperClase` , pero además va a tener ese atributo y ese método propios a ella.

▼ **Overriding:** permite que una subclase implemente una versión especializada de un método que es heredado desde una superclase.

- Basta con volver a definir el método, usando el mismo nombre que tiene en la superclase.

- El método `super()` permite la implementación de un método de la superclase sin nombrar explícitamente a la clase superior, de la forma `super().metodo(argumentos)`
  - Es una buena práctica para mejorar la lectura y evitar errores por cambiar el nombre de la superclase o multi-herencias.

▼ Se puede crear una subclase a partir de una clase **built-in** de Python

```
class ListaDeContactos(list):
        """
    Tiene todos los métodos de las listas
    y agrega el método 'buscar'
    """
    def buscar(self,nombre):
        matches = []
        for contacto in self:
              if nombre in contacto.nombre:
                  matches.append(contacto)
              return matches
```

Polimorfismo

- **Polimorfismo:** propiedad por la que es posible enviar mensajes sintácticamente iguales a objetos de tipos distintos.
  - Se trata de utilizar objetos de distinto tipo con la misma interfaz.
  - Mecanismos típicos para proveer polimorfimo

- **Overriding**

▼ **Overloading**

- Capacidad de definir un método con el mismo nombre, pero con distinta cantidad y tipo de parámetros.
- Capacidad de una función de ejecutar **distintas acciones** **dependiendo del tipo y número de argumentos** que recibe.

⚠️ Python no soporta *function overloading*
- Solo se puede "simular" usando algunos parámetros con valores por defecto o números de parámetros variables.
- Si uno define la misma función múltiples veces, Python sólo conserva la última versión escrita.

Overloading de operadores en Python

- Existen muchos operadores en Python que funcionan de forma distinta para cada clase built-in.
  - ej. el operador `+` suma números, pero también concatena strings y mezcla listas.

▼ Es posible personalizar la mayoría de operadores, modificando sus métodos respectivos...

Operator Magic Methods	Description
__add__(self, other)	To get called on add operation using + operator
__sub__(self, other)	To get called on subtraction operation using - operator.
__mul__(self, other)	To get called on multiplication operation using * operator.
__floordiv__(self, other)	To get called on floor division operation using // operator.
__truediv__(self, other)	To get called on division operation using / operator.
__mod__(self, other)	To get called on modulo operation using % operator.
__pow__(self, other[,modulo])	To get called on calculating the power using ** operator.
__lt__(self, other)	To get called on comparison using < operator.
__le__(self, other)	To get called on comparison using <= operator.
__eq__(self, other)	To get called on comparison using == operator.
__ne__(self, other)	To get called on comparison using != operator.
__ge__(self, other)	To get called on comparison using >= operator.

`__repr__` vs `__str__`

- Retornan una representación en texto (string) para ser usado por `print` .
  - Si se implementa ambos, `print` utiliza `__str__`
- `__str__` está enfocada en una representación **legible** del objeto (nivel "usuario"), mientras que `__repr__` es una representación **completa** (nivel "desarrollador").

▼ `__repr__` puede ser usado para mostrar los elementos de una lista

- Si implementamos solamente `__str__` en nuestra clase, al imprimir una lista de objetos no nos va a salir lo que deseamos → tenemos que implementar `__repr__` para imprimir una lista de objetos.

Duck typing

> *"If it walks like a duck and quacks like a duck then it is a duck"*
> - no importa de qué tipo sea un objeto mientras contenga la acción

▼ Python utiliza un sistema de tipos dinámicos, lo que permite que el tipo de una variable se determine al

momento de ejecutar el código (y no al compilarlo ni al escribirlo).

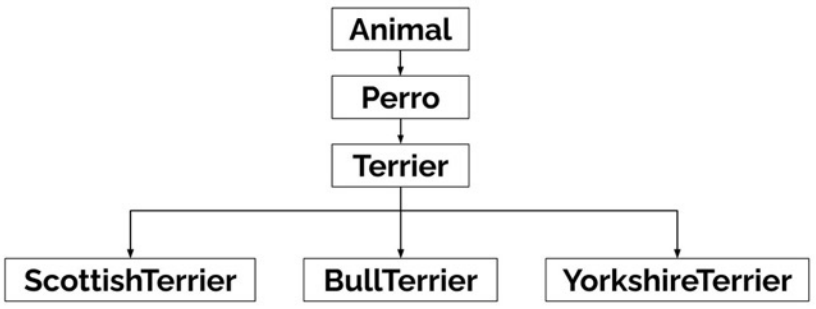
```
    def activar(pato):  #
        pato.gritar()   #
        pato.caminar()

    donald = Pato()
    elon = Persona()
    activar(donald)
    activar(elon)
]
```

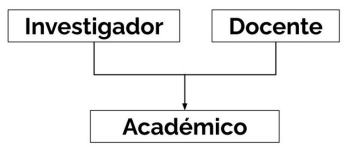
```
Quack!
Caminando como un pato
¡Ahhh!
Caminando como un humano
```

Si se le pasara como argumento una variable que no tiene esos métodos, entonces sí habría un error.

Multiherencia



Herencia simple



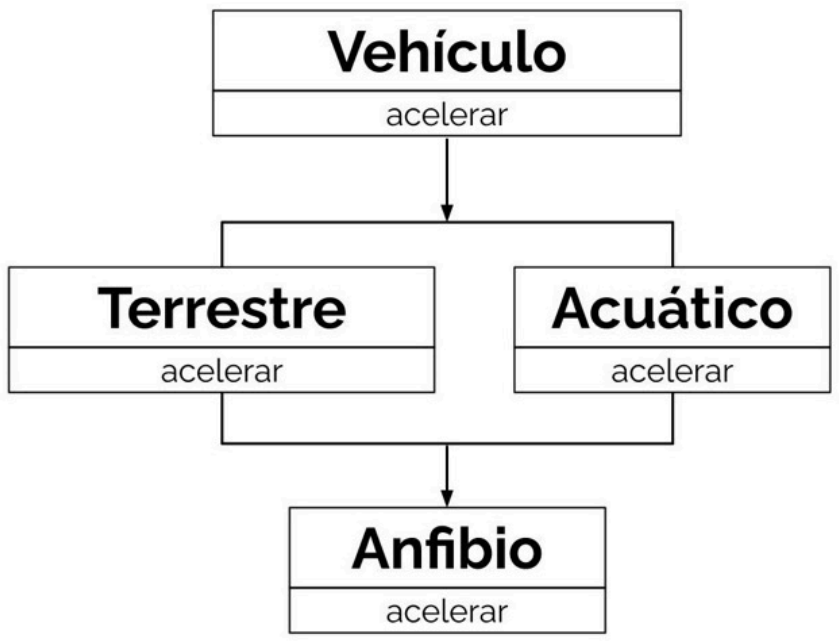
Multiherencia

- **Multiherencia:** una subclase puede heredar desde más de una superclase
  - Sigue la forma `class SubClase(SuperClase1, SuperClase2):`
- **Ventajas:** flexibilidad, reutilización de código y representación de múltiples roles.
- **Desafíos:** ambigüedades/conflictos, MRO (Orden de Resolución de Métodos) y complejidad
- Cuando más de una clase base tienen un método con el mismo nombre, el intérprete de Python decide cuál llamar siguiendo el **MRO**
  1. Busca en la clase derivada
  2. Busca en la primera clase base
  3. Busca en las clases restantes

Problema del diamante

- Existe una jerarquía de diamante cada vez que tenemos más de un “camino” en la jerarquía desde la clase inferior a una clase superior.

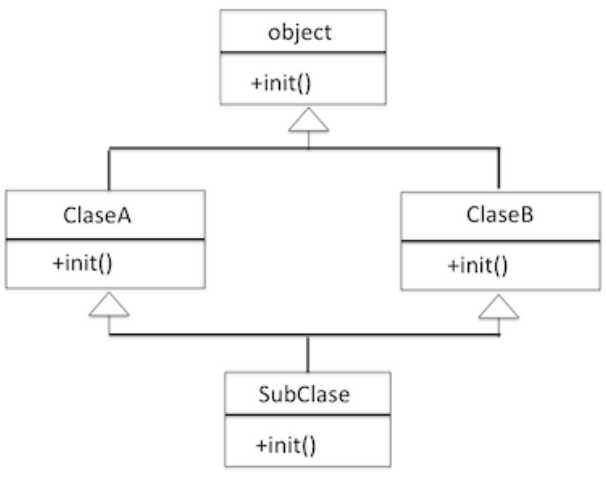
▼ Puede ocurrir por dos superclases que derivan de la misma clase



`Anfibio` implementa `acelerar` , que llama a otro método del mismo nombre en `Terrestre` y `Acuático` . Además, cada una de estas últimas clases también llaman a `acelerar` con respecto a su superclase, `Vehículo.` Así, una llamada a `acelerar` en `Anfibio` produce dos llamadas a `Vehículo`

▼ o simplemente porque **una clase deriva de dos clases**, sin necesidad de una tercera superclase.

- Esto ocurre porque Python tiene una clase superior `object` de la cual heredan **todas** las clases que creamos.



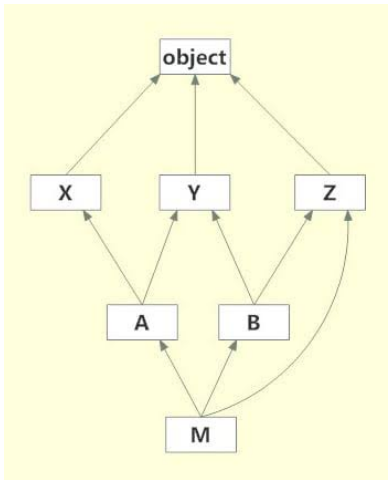
Si en la SubClase modificamos el inicializador de tal forma que llama a los iniciadores de las superclases como `ClaseA.__init__()` y `ClaseB.__init__()` , entonces se va a **inicializar 2 veces la clase** `object` ❎❎

- Dependiendo del código, esto podría hacer que se produzcan **acciones repetidas** en la clase superior.
- Para evitar errores, la clase debe preocuparse de llamar a inicializar a la clase que la “precede” en el orden del esquema de multiherencia. Esto se logra usando `super()`
  - El orden de las clases es **izquierda** → **derecha** dentro de la lista de superclases desde donde hereda la subclase.
  - En otras palabras, `super()` llama a la clase que viene antes según el orden que muestra `__mro__` .

Método `__mro__`

- Significa ***method resolution order.***
  - Muestra el orden en la jerarquía de clases a partir de la clase actual.
  - Se basa en el algoritmo C3.

▼ Analiza el MRO de este ejemplo



▼ Solución

`M -> A -> X -> B -> Y -> Z -> object`

- Puede lanzar `TypeError` si se crea un esquema de **multiherencia inválido**, es decir, donde no se puede crear un MRO consistente.

▼ Ejemplo

```python
class X():
    def call_me(self):
        print("soy X")

class Y():
    def call_me(self):
        print("soy Y")

class A(X, Y):
    def call_me(self):
        print("soy A")

class B(Y, X):
    def call_me(self):
        print("soy B")

class F(A, B):
    def call_me(self):
        print("soy F")
```

▼ Solución

```python
class X():
    def call_me(self):
        print("soy X")

class Y():
    def call_me(self):
        print("soy Y")

class A(X, Y):
    def call_me(self):
        print("soy A")

class B(X, Y):
    def call_me(self):
        print("soy B")

class F(A, B):
    def call_me(self):
        print("soy F")

print(F.__mro__)
```

Cómo implementar correctamente multiherencia

- **Consideraciones**
  - Si ocupamos `super()` en una clase A que viene antes de una clase B según el MRO, esta va a referenciar dicha clase B.
  - Para manejar los parámetros que se le pasan al inicializador de una clase con múltiples superclases, se pueden ocupar diccionarios (`**kwargs`)
- **Protocolo**

▼ (1) **Declara los atributos propios** de cada clase en su respectivo inicializador (`__init__`). Además, en cada

uno de estos, agrega un “captador” para una cantidad variable de argumentos por palabra clave (`**kwargs`)

```
class Investigador:
        def __init__(self, area, **kwargs):
        ...
class Docente
        def __init__(self, departamento, **kwargs):
        ...
class Academico(Docente, Investigador):
        def __init__(self, nombre, oficina, **kwargs): #1
        ...
```

▼ (2) Dentro de la definición de cada inicializador, **asigna los valores a los atributos propios.** Además, ocupa `super().__init__(**kwargs)` para pasarle los otros kwargs a la clase que siga en el MRO.

```
class Investigador:
        def __init__(self, area, **kwargs):
        super().__init__(**kwargs)
        self.area = area

class Docente
        def __init__(self, departamento, **kwargs):
        super().__init__(**kwargs)
        self.departamento = departamento

class Academico(Docente, Investigador):
        def __init__(self, nombre, oficina, **kwargs): #1
        super().__init__(**kwargs)
        self.nombre = nombre
```

- (3) Cuando uno instancie la clase lo mejor es hacerlo a través de puros `kwargs` . De esta forma, el intérprete de Python se va a encargar de formar el diccionario con estos argumentos, recorrer la cadena de clases según el MRO y pasarle a cada clase el atributo que le corresponde. Finalmente, cuando llegue a la última clase, le va a pasar un diccionario **vacío** a la clase `object`, lo cual está correcto ya que no acepta ningún argumento.

- ⚠️ **Con respecto al paso (3)**
  - Si instancias la clase con la **cantidad exacta de argumentos** (para el total de atributos entre todas las clases que están en la cadena del MRO), entonces el objeto de la clase se va a crear correctamente.
  - Si le pasas menos o más argumentos, te va a lanzar error.
    - Si ocupas **valores por defecto,** podrías evitar errores por pasarle **menos argumentos** de los esperados.

▼ Si creas una **clase** `Raíz` **,** de tal forma que sea la última en la cadena del MRO y que capte todos los argumentos que sobran, podrías evitar que le lleguen **argumentos sobrantes** a `object`

```
class RaizSegura:
    """Clase base que intercepta kwargs sobrantes antes de que lleguen a obje
ct."""
    def __init__(self, **kwargs):
        # Si llegaron kwargs hasta aquí, significa que nadie los reclamó.
        if kwargs:
            # Opción 1: Levantar un error claro
            raise TypeError(f"Error: Se enviaron argumentos inesperados que n
adie usó: {kwargs}")

            # Opción 2: Solo imprimir una advertencia (si prefieres ser permi
sivo)
```

```
            # print(f"⚠ ADVERTENCIA: Argumentos ignorados: {kwargs}")

        # ¡La magia ocurre aquí! Llamamos a object sin argumentos.
        super().__init__()
```

Clases abstractas

- **Clases abstractas**: clases cuya intención no es ser instanciada, sino que **sirven para modelar otras clases.**
  - Contienen 1 o más **métodos abstractos**, los cuales **deben ser sobre-escritos** (implementados) **por sus subclases.**

▼ Para crear una clase abstracta se ocupa la clase `ABC` y el decorador `abstractmethod` , ambos provenientes del módulo `abc`

```
from abc import ABC, abstractmethod


class Base(ABC):          ## al heredar de ABC, la clase Base se define como abstra
ct

    @abstractmethod       ## Esto hace que metodo_1 sea abstracto. Una subclase de
Base está obligada a implementarlo.
    def metodo_1(self):
        pass

    @abstractmethod
    def metodo_2(self):
        pass

 # Una subclase válida* se vería como sigue
 # *implementa por lo menos todos los métodos abstractos de la superclase abstracta
 class SubClase1(Base):

    def metodo_1(self):
        pass

    def metodo_2(self):
        pass
```

- Las subclases no necesitan tener sus propias definiciones de los métodos abstractos, también pueden ocupar la definición de la clase abstracta usando `super()`
- Si bien es obligatorio que una subclase implemente todos los métodos abstractos, esta última también puede definir sus propios métodos e incluso ocupar métodos no-abstractos de la clase abstracta.

📌 **Si una subclase A** de una clase abstracta B **no se va a instanciar,** no necesita implementar todos los métodos abstractos de la última. En su lugar, si esta subclase le hereda a otra clase C, entonces esta última sí tendría que implementar todos los métodos abstractos para poder instanciarse.
En particular, es posible que la clase A implemente algunos métodos abstractos y C implemente el resto, de modo tal que A no sea capaz de instanciarse, pero C sí lo sea.

Decoradores

- Un **decorador** es una función (callable o invocable) que toma otra función como *input* y retorna una versión modificada de esta última.
- Permiten modificar el comportamiento de una función, sin cambiar las líneas de código que la componen.

Objetos de primera clase

- Las funciones en Python son **objetos de primera clase**, por lo que
  - Puedes asignar funciones a variables

  ▼ Puedes ocupar funciones como argumentos para otras funciones

```python
# función regular que espera un nombre como string y retorna un string
def decir_hola(nombre: str) -> None:
    return f"Hola {nombre}"

# función que espera una función como argumento y retorna una función
def saludar_dani(funcion_saludadora):
    return funcion_saludadora

saludar = saludar_dani(decir_hola)
print(saludar("Dani"))
```

  - Puedes almacenar funciones en estructuras de datos
  - Tienen atributos (puedes agregar o acceder dinámicamente a los atributos de una función)

> 👁️ **Ojo**
> - Un nombre de función sin paréntesis es una **referencia** a una función
> - Un nombre de función con paréntesis finales la **llama** y hace referencia a su valor de **retorno**.

Sintaxis de los decoradores

▼ Se pone el símbolo `@` seguido del nombre del decorador, es decir, `@nombre_decorador` . Luego, en la línea de abajo empieza la definición de la función que recibe como argumento.

```python
#Cuando vemos este código:
@decorador
def mi_funcion():
    pass

######

# Es realmente una abreviación para:
def mi_funcion():
    pass

mi_funcion = decorador(mi_funcion)
```

▼ **Definir un decorador D** sigue el mismo formato que la definición de cualquier otra función, solo que en este caso (1) recibe una función A como argumento y (2) define otra función B que modifique a A, de tal modo que D retorna B.

```python
def agregar_exclamacion(func):
    def decorada():
        return func() + "!"
    return decorada

@agregar_exclamacion
def decir_hola():
    return "Hola"
```

```
print(decir_hola())
```

✅ Las **funciones internas/anidadas** (aquellas definidas al interior de otra función) solo existen dentro de la función externa

Función `property` y decorador `@property`

- En el paradigma OOP, si uno quiere trabajar con atributos privados, se deben definir métodos específicos para obtener el valor de estos (**getters**) y para actualizar el valor de estos (**setters**).
- `property(get, set, del)` es una función built-in que crea un atributo administrado en una clase.
  - Permite definir un getter, setter y deleter
- La función se llama como `property(fget=None, fset=None, fdel=None)` , donde:
  - `fget` capta el atributo.
  - `fset` setea el atributo (le asigna un valor)
  - `fdel` borra el atributo
- Si uno quiere ocupar directamente la **función** `property` para implementar los métodos para un atributo privado, debe poner una línea de la forma `variable = property(a,b,c)` en la definición de la clase

  ▼ Donde `variable` es el nombre público que va a recibir el atributo afuera de la clase y `a,b,c` son nuestras implementaciones de las funciones respectivas.

  Considere que una instancia es `pepa` :

  - `a` se llama cuando accedemos al valor del atributo ⟶ `pepa.variable`
  - `b` se llama cuando modificamos el valor del atributo ⟶ `pepa.variable = 8`
  - `c` se llama cuando borramos el atributo ⟶ `del pepa.variable`

  ▼ Ejemplo

```
class Circulo:

    def __init__(self, radio):
        self._radio = radio  # atributo a proteger, privado

    # Getter para el radio
    def get_radio(self):
        print("Obteniendo el radio...")
        return self._radio

    # Setter para el radio, que asegura valores positivos
    def set_radio(self, value):
        if value >= 0:
            self._radio = value
        else:
            print("El radio no puede ser negativo")

    # Metodo que retorna el area del circulo
    def area(self):
        print("Obteniendo el área...")
        print("Area:",3.14 * self._radio ** 2)

    # Definimos propiedades usando property()
    radio = property(get_radio, set_radio)

c = Circulo(5)
```

```
print("Radio:", c.radio)
c.area()

# Intentamos cambiar el radio a un valor negativo
c.radio = -3
c.area()

# Cambiamos el radio a un valor válido
c.radio = 10
c.area()
```

- Alternativamente, se puede ocupar el decorador `@property` para **definir un método como un atributo de solo lectura.**
  - Se pone `@property` arriba de la definición de la función getter. El nombre de dicho método (ej. `nombre_publico`) va a ser el nombre que se va a utilizar para acceder al atributo privado afuera de la clase.
  - Luego, las implementaciones del setter y el deleter ocupan decoradores de la forma `@nombre_publico.setter` y `@nombre_publico.deleter`

▼ Ejemplo

```
class Email2:

    def __init__(self, address):
        self._email = address

    @property
    def email(self):
        return self._email

    @email.setter
    def email(self, value):
        if '@' not in value:
            print(f"El string {value} no parece una dirección de correo.")
        else:
            self._email = value
            # el setter le asigna un valor al "atributo"

mail = Email2("profe1@gmail.com")
print(mail.email)
mail.email = "profe2@gmail.com"
print(mail.email)
mail.email = "profe2.com"
```

📌

▼ **No es necesario poner el atributo privado dentro del constructor** `__init__` . Si uno lo pone en la definición del setter y en el `__init__` le asigna el argumento a la property, entonces se llama el setter y

se le asigna por interno ese valor al atributo privado.

```
class Vehiculo(ABC):
    identificador = 0
    def __init__(self, rendimiento: int, marca: str, energia: float = 111.5,
*args, **kwargs) -> None:
        super().__init__(*args, **kwargs)
        self.rendimiento = rendimiento
        self.marca = marca
        self.energia = energia
        self.identificador = Vehiculo.identificador
        Vehiculo.identificador += 1

    @property
    def autonomia(self) -> float:
        kilometros = self.energia * self.rendimiento
        return float(kilometros)
```

```
    @property
    def energia(self) -> float:
        return self.energia_privada

    @energia.setter
    def energia(self, valor) -> None:
        if valor < 0:
            self.energia_privada = 0
        else:
            self.energia_privada = round(valor, 1)
```

El atributo privado `energia_privada` se asigna a través del setter de la property `energia` . Así, en el constructor se le pasa el argumento a la property, para que se llame al **setter** y se le asigne el valor a `energia_privada` . **El nombre del argumento no necesita llamarse igual que la property.**

- Sin embargo, es **buena práctica** ponerlo para especificar la existencia de dichos atributos privados.

Métodos estáticos

📌 Algunos idiomas OOP (no incluye Python) no permiten definir métodos de clases que no utilicen `self` . Así mismo, **PEP8** se opone a definir un método que tome como argumento `self` , pero no lo utilice en su interior.

- Son métodos que pertenecen a una clase, pero que no dependen de la información de la clase ni de atributos de la instancia.
  - → no requieren ningún atributo principal como sucede con los métodos normales, o sea, **no requiere** `self`
- Para declarar un método estático, solo necesitamos usar el decorador `@staticmethod` y nuestro método sin el `self`
- Luego, para ocupar un método estático afuera de la clase basta con usar `Clase.metodo_estatico()`

▼ Ejemplo

```
class MiMath:

    @staticmethod
    def sumar(num1, num2):
        return num1 + num2
```

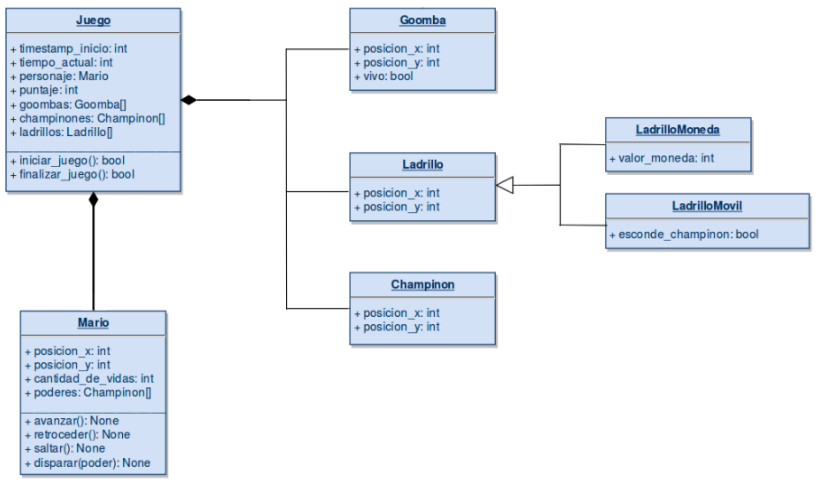
```python
    @staticmethod
    def restar(num1, num2):
        return num1 - num2

print(MiMath.sumar(5, 6))
print(MiMath.restar(14, 1))
```

Diagrama de clases

- Herramienta que permite visualizar fácilmente tanto las clases que componen un sistema, así como también sus atributos, métodos y las interacciones que existen entre ellas.
- Pertenece al conjunto de herramientas provistas por el Lenguaje de Modelado Unificado (**UML**).

Elementos de un diagrama de clases



Ejemplo de diagrama de clases para representar el juego de Super Mario

Clases

- Se representan por un rectángulo dividido en tres niveles:
  1. Nombre de la clase
  2. Atributos → especificar nombre y tipo de variable
  3. Métodos → especificar nombre, parámetros que recibe y tipo de variable esperado para su retorno

▼ Ejemplo

Auto
+ dueño: str or None
+ marca: str
+ modelo: str
+ _kilometraje: int
+ @kilometraje: int (getter y setter)
+ conducir(kms): None
+ vender(dueño): Str

📌 UML no tiene incorporada la modelación de *properties.* Sin embargo, el curso ocupa la siguiente **convención**:

- `_atributo: int` representa un **atributo privado** de tipo int

- `@atributo: int (getter y setter)` indica una **property** de tipo int y tiene tanto el getter como el setter implementados.

Relaciones

- Relaciones de **contención**
  - Un objeto contiene a otra clase como atributo.
  - Estrictamente existen 2 tipos

    ▼ **Composición** (rombo ⧫)

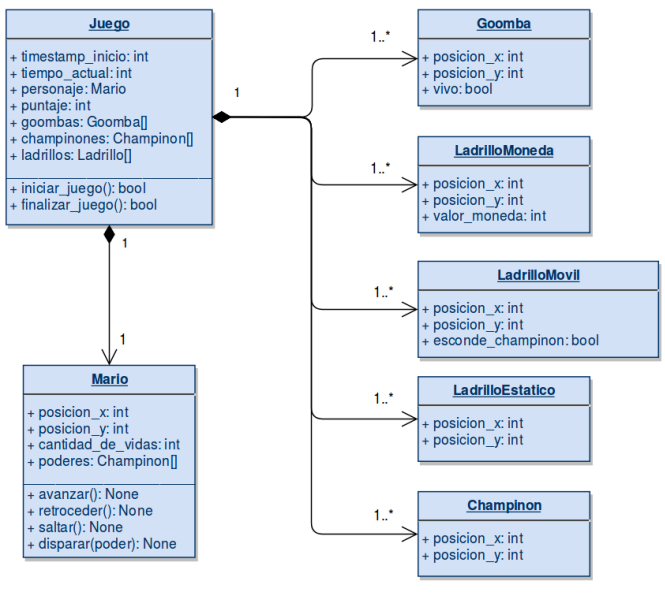
    - El tiempo de vida del objeto que componemos está **condicionado** por el tiempo de vida del objeto que lo incluye.
    - ej. una clase `Mario` no sigue si eliminamos la clase `Juego`

    ▼ **Agregación** (rombo ◊)

    - El tiempo de vida del objeto que agregamos es **independiente** del tiempo de vida del objeto que lo incluye.
    - ej. una clase `Bowser` tiene un atributo `self.goombas = []` que completa con una cadena de objetos de clase `Goomba` . Si muere `Bowser` , no necesariamente mueren los `Goomba`

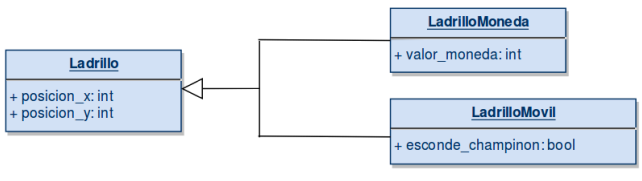
  - Para el curso no se ocupa esa distinción entre rombos. En otras palabras, una relación de contención se va a denotar por ⧫ o ◊.

▼ Ejemplo de diagrama de clases



- Relaciones de **herencia**
  - Una subclase hereda los atributos y métodos de una superclase.
  - Se representa por una flecha que apunta hacia la superclase (▷)
    - En las subclases solo se ponen los métodos/atributos propios, o sea, no se repiten los heredados.

  ▼ Ejemplo



Metaclases

- La razón por la que elementos se pueden guardar en variables y entregar como argumentos es porque tienen un tipo de dato, es decir, **son instancias de una clase.**

    - Los enteros son instancias de la clase `int`
    - Las listas son instancias de la clase `list`
- Una **metaclase** es una clase cuyas instancias son otras clases.
- El comando `type()` recibe un argumento y retorna su tipo de dato, es decir, la clase a la que pertenece la instancia.
    - Si no se especifica la metaclase en la definición de una clase, se toma la metaclase por defecto en Python, o sea, `type`

▼ Para **declarar que una clase tiene cierto tipo** se añade el argumento por palabra clave `metaclass=tipo_de_clases`

```
class Gato(metaclass=type):
    def __init__(self, nombre):
    self.nombre = nombre
```

Creación de metaclases

- Las metaclases se declaran como clases que heredan de `type`
- **Principal utilidad:** controlar la creación e instanciación de una clase.

▼ Usos específicos

- Clases que no pueden tener subclases
- Clases que queremos instanciar una única vez

Método `__new__`

```
class MetaClaseImpresora(type):
        def __new__(cls, nombre, clases_base, diccionario):
                # Creamos la clase usando el __new__ de type
                clase = super().__new__(cls, nombre, clases_base, diccionario)
                # Retornamos la clase creada
                return clase
```

Implementación básica de `__new__` . Si queremos que se **cree** la clase, debemos llamar al `__new__` de `type` y retornar la clase.

- Permite controlar la **creación de una clase.**

▼ Parámetros

- `cls` representa la metaclase actual
- `nombre` nombre de la clase
- `clases_base` tupla con las clases de las que hereda la clase instanciada
- `diccionario` contiene la información de la clase instanciada

▼ Ejemplos de uso

```
class MetaFinal(type):
        def __new__(cls, nombre, clases_base, diccionario):
                # Revisamos las clases base de la clase que se está creando
                for clase in clases_base:
                        # Si la clase es instancia de la metaclase, no puede ser cr
eada
                        if isinstance(clase, cls):
                                print(f'No puede crearse la clase {nombre}')
                                return None
                print(f'Clase {nombre} creada correctamente')
                return super().__new__(cls, nombre, clases_base, diccionario)

class NoHeredable(metaclass=MetaFinal):
      def __init__(self, *args, **kwargs):
              super().__init__(*args, **kwargs)

class ClaseImposible(NoHeredable):
        def __init__(self, *args, **kwargs):
                super().__init__(*args, **kwargs)
```

Implementación de `__new__` que permite crear una clase que no puede heredarse, es decir, **no puede tener subclases.** En particular, consiste en crear una metaclase que defina este comportamiento para las clases que no son heredables. Luego, si uno crea una clase de este estilo (no-heredable) y trata de crear otra clase que la herede, entonces va a ocurrir un error.

```
class MetaTransformaPaladin(type):
        def __new__(cls, nombre, clases_base, diccionario):
                # Cambiamos el nombre de la clase
                nuevo_nombre = 'Paladin'

                # Cambiamos las clases base
                nuevas_clases_base = (Guerrero,)

                # Actualizamos el diccionario con el nuevo atributo
                diccionario['defensa'] = 100

                # Llamamos al __new__ de type con los nuevos argumentos
                clase = super().__new__(cls, nuevo_nombre, nuevas_clases_base, dicc
ionario)
                return clase
```

Implementación de `__new__` que permite **cambiar el nombre de la clase, las clases de las que hereda y agrega un atributo**

Método `__init__`

- Permite controlar la **instanciación de una clase.**
  - Ocupa los mismos parámetros que `__new__` , pero en lugar de `cls` ocupa `self`
  - No permite modificar cosas propias de la clase como su nombre o sus clases base.

▼ Podrías implementar una metaclase como `__eq__`

```
class MetaClaseImpresoraInit(type):
        def __init__(self, nombre, clases_base, diccionario):
                # Creamos la clase usando el __init__ de type
                clase = super().__init__(nombre, clases_base, diccionario)
```

```
                # Retornamos la clase creada
                return clase
```

▼ Ejemplos de uso

```
class MetaAgregaDefensa(type):
        def __init__(self, nombre, clases_base, diccionario):
                # Actualizamos el diccionario agregando el atributo defensa
                self.defensa = 100
                clase = super().__init__(nombre, clases_base, diccionario)
                return clase
```

Implementación de `__init__` que se encarga de instanciar la clase y agregarle un atributo

Creación de instancias

- Un ***callable*** es un objeto cuyo método `__call__` está implementado.
- Cuando uno crea una instancia hace `NombreClase()` , de modo que se llama a `__call__` de la **metaclase**, que luego se encarga de llamar al `__init__` de la **clase** y así crear la instancia correctamente.
  - ⟶ Para modificar la creación de instancias, uno puede **modificar el método** `__call__` en la metaclase.
- `__call__` recibe como argumentos a `cls` y un número variable de argumentos (que depende de cada clase) que pueden ser organizados usando `* args` y `** kwargs`

▼ Ejemplos de uso

```
class MetaImprimeInstancias(type):
        def __call__(cls, *args, **kwargs):
                # Opcional:
                print('LLamando a __call__ de MetaImprimeInstancias')
                print(f'Instanciando la clase {cls.__name__} con args: {args} y kwa
rgs: {kwargs}')

                # Clave:
                instancia = super().__call__(*args, **kwargs) #1
                return instancia
```

Metaclase que muestra cuando se crea una instancia. (1) Esa línea permite **crear la instancia.** Llama al método `__call__` de `type` , el cual se encarga de manejar la memoria y llamar al `__init__` de la clase.

```
class MetaSoloUna(type):
        def __call__(cls, *args, **kwargs):
                if not hasattr(cls, 'instance'):
                        cls.instance = super().__call__(*args, **kwargs)
                return cls.instance
```

Metaclase que sólo permite una y sólo una instancia de una clase (**singletón**). Se aprovecha de los atributos de clase, para crear uno cuyo valor sea la instancia única. Así, si al llamar a la metaclase no tiene el atributo, eso significa que no se ha instanciado todavía y, por ende, debe crearse el atributo y asignarle la instancia. Luego, retorna la instancia.